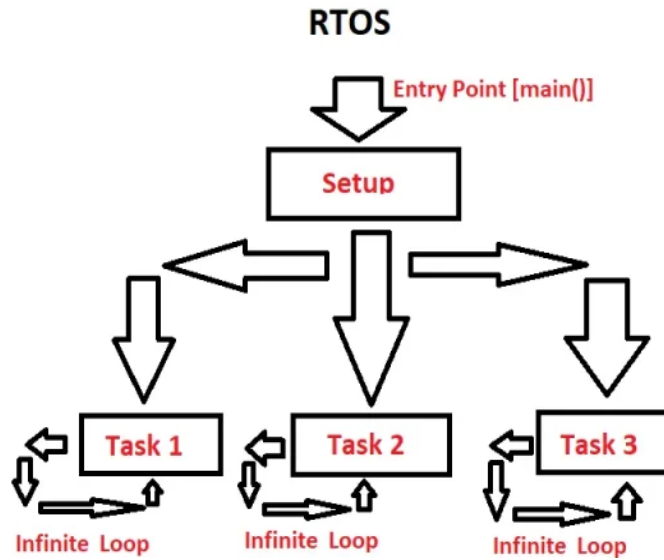
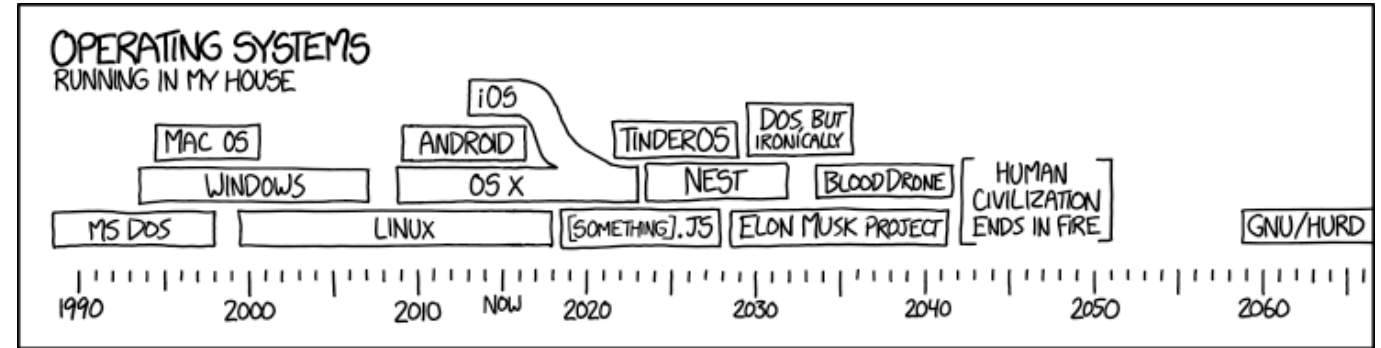




<https://embeddedthere.com>



<https://xkcd.com/722/>

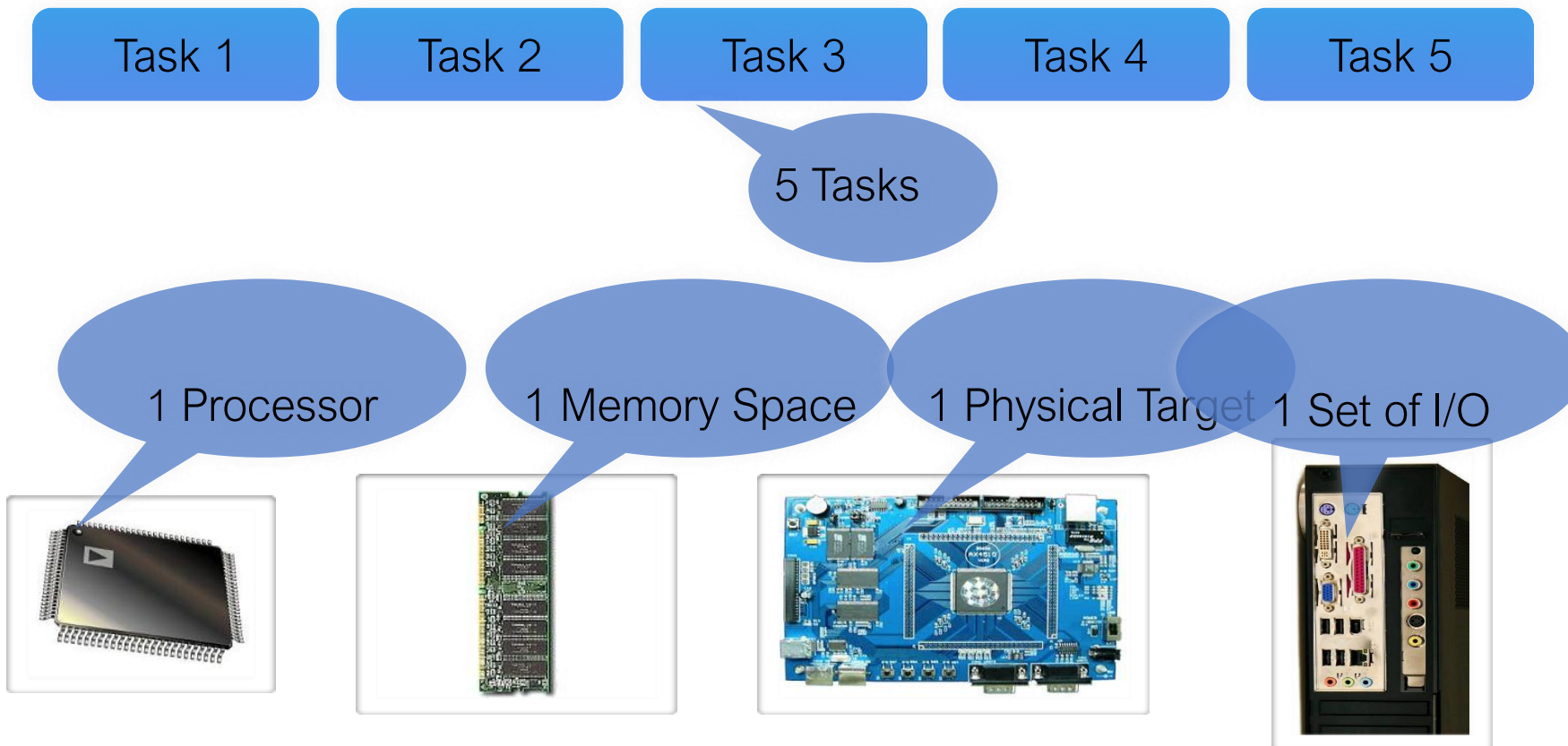
Special Topic - RTOS

(3A) RTOS Implementation

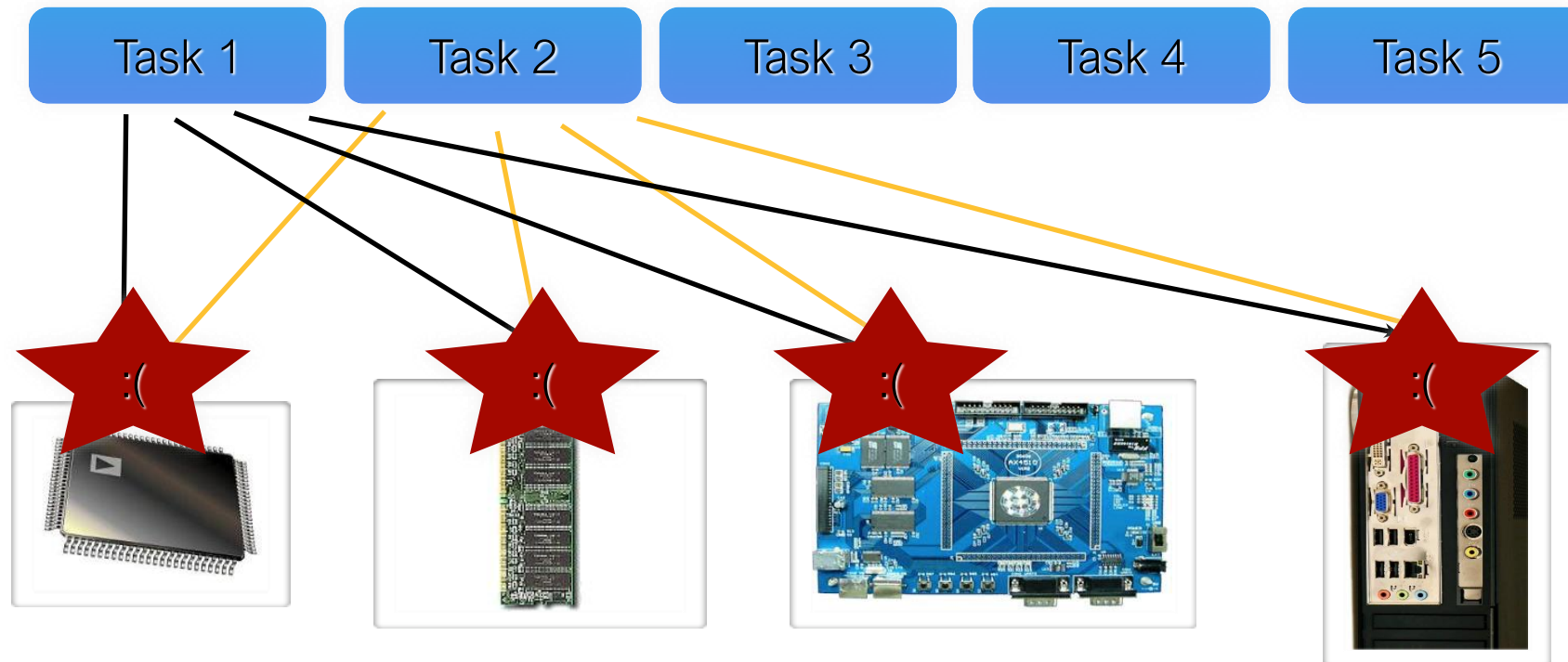
Real Time Operating System (RTOS)

- It is an OS that is commonly used with Embedded Systems
- Can respond quickly without too much latency to interrupts.
- Standard OS'es use queues which has a slower response time.
- An RTOS is just a program that is run from startup of the processor. It is launched from the RESET vector.
- An RTOS is multitasking with each task being a function that is to be performed in parallel (pseudoparallel on 1 processor systems).
- An RTOS will a scheduler which will control which task is run and what time that task is run.

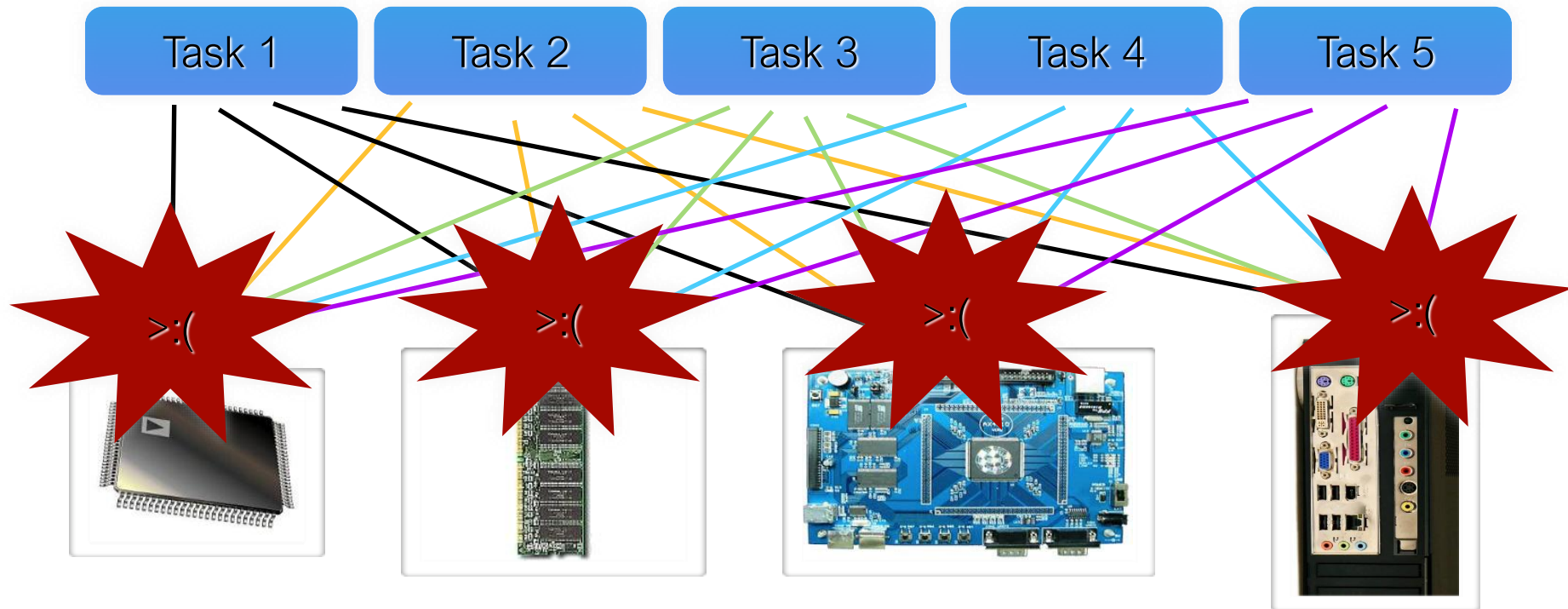
What it's all about: Why?



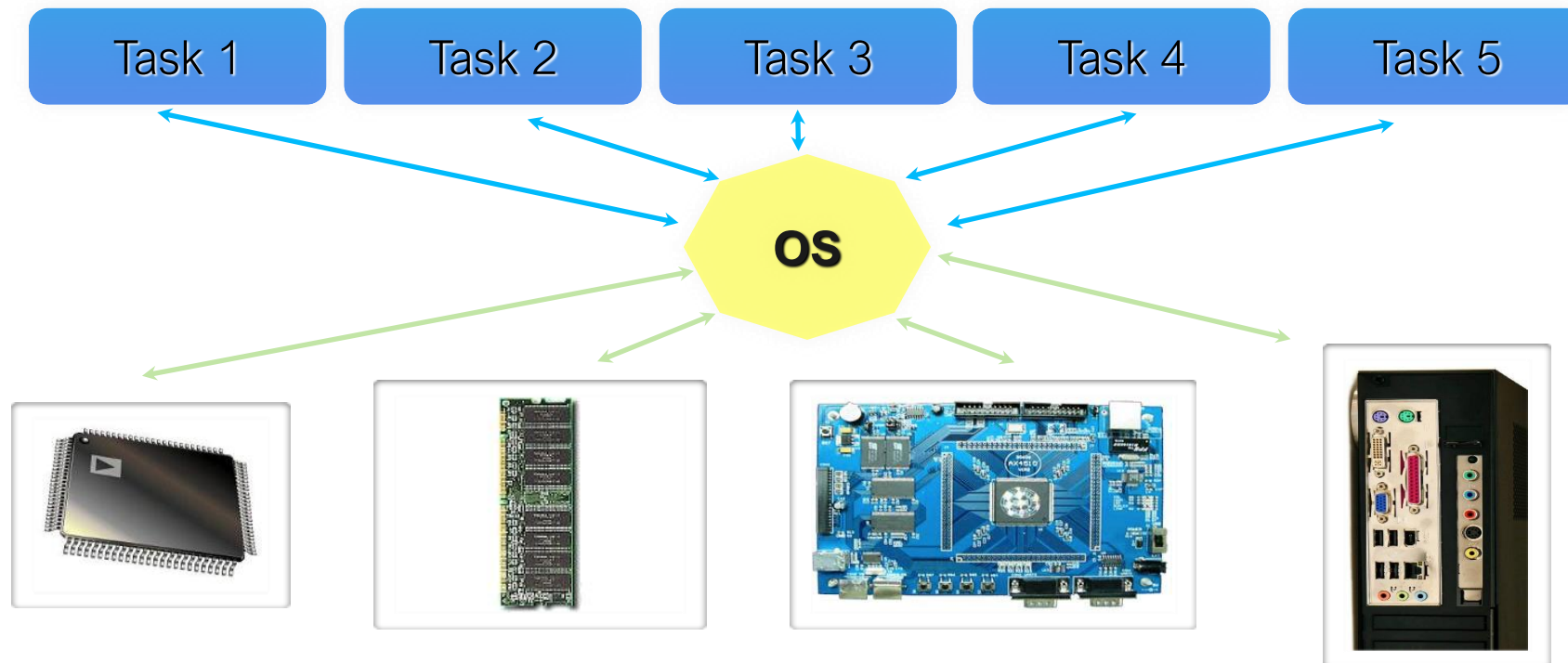
What it's all about: Why?



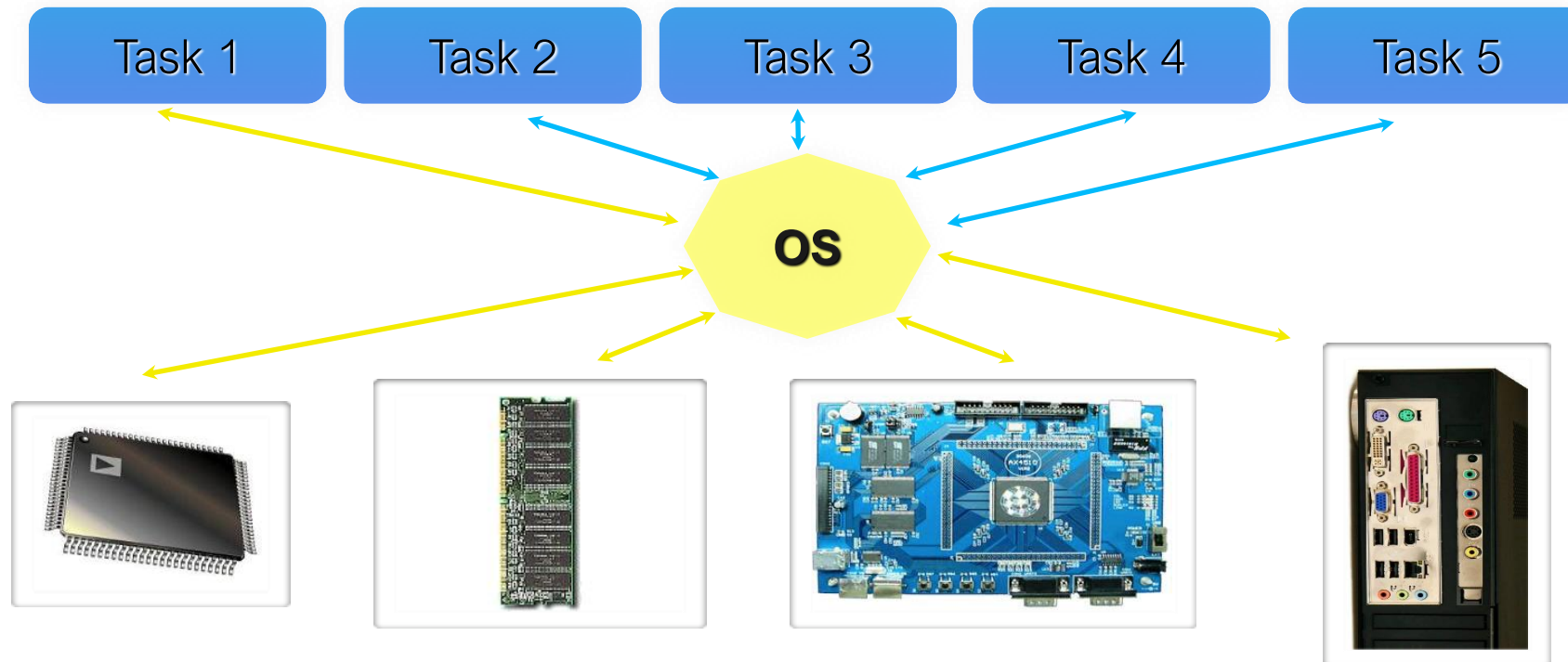
What it's all about: Why?



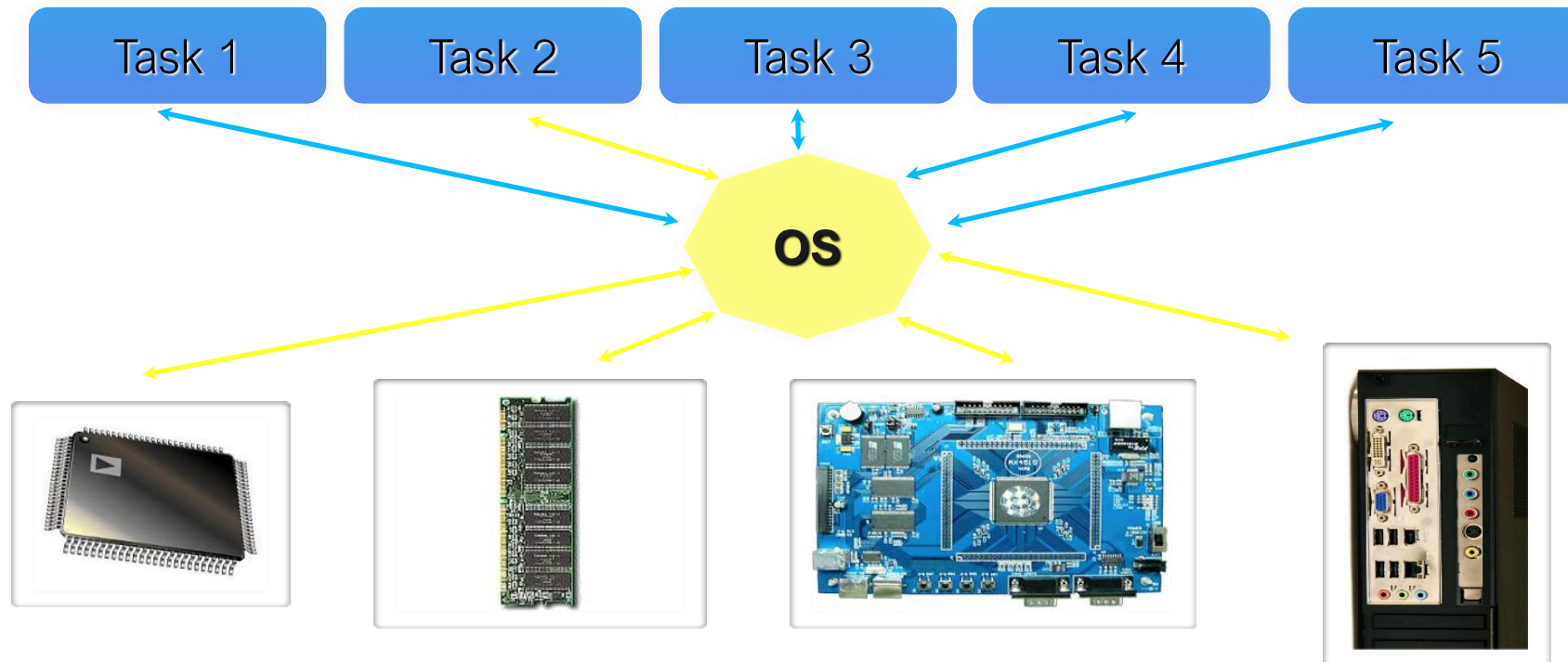
What it's all about: Why?



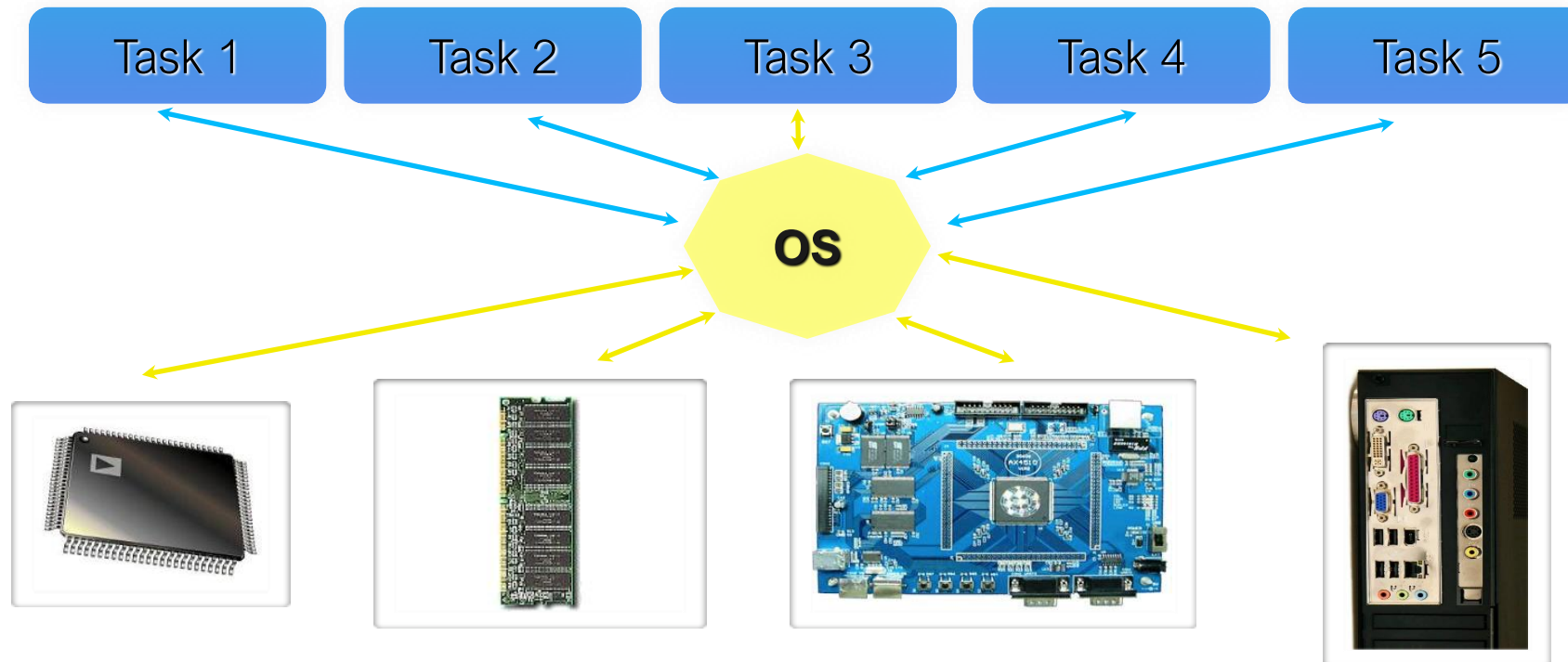
What it's all about: Why?



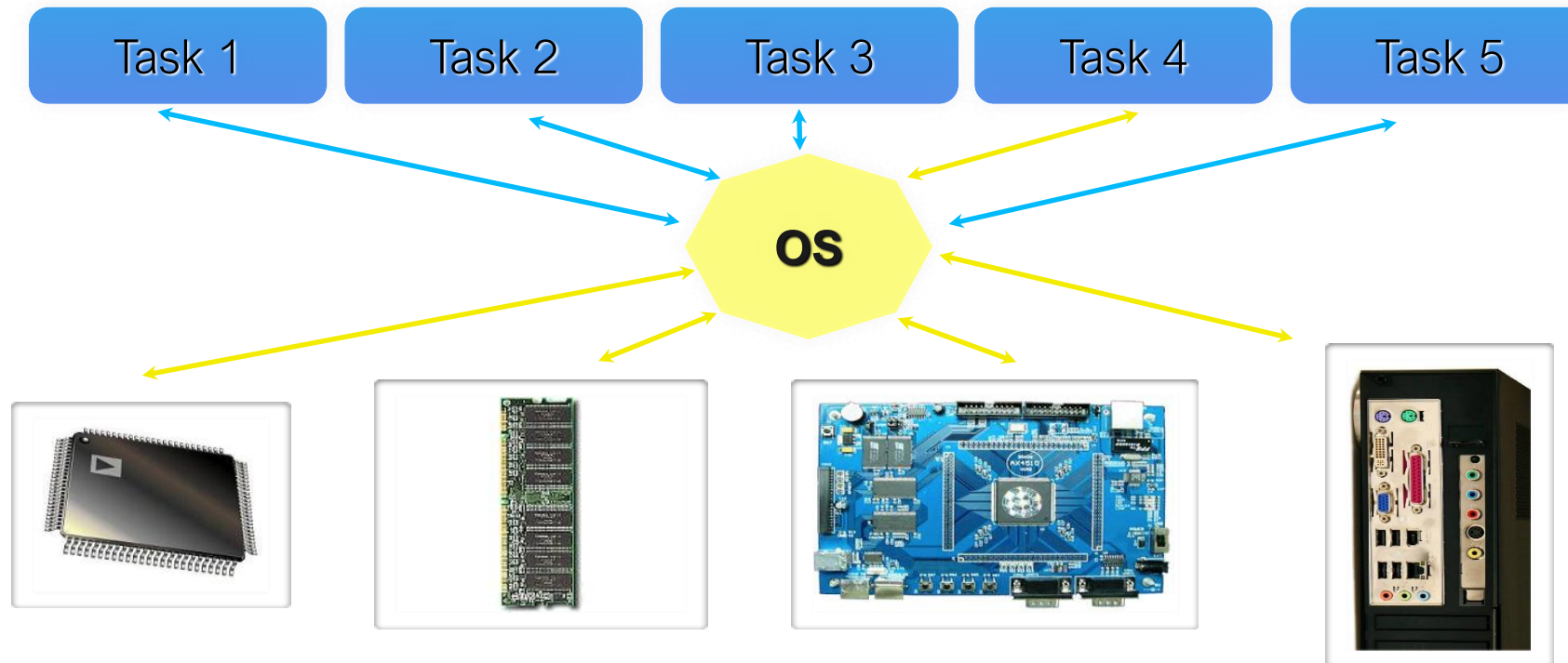
What it's all about: Why?



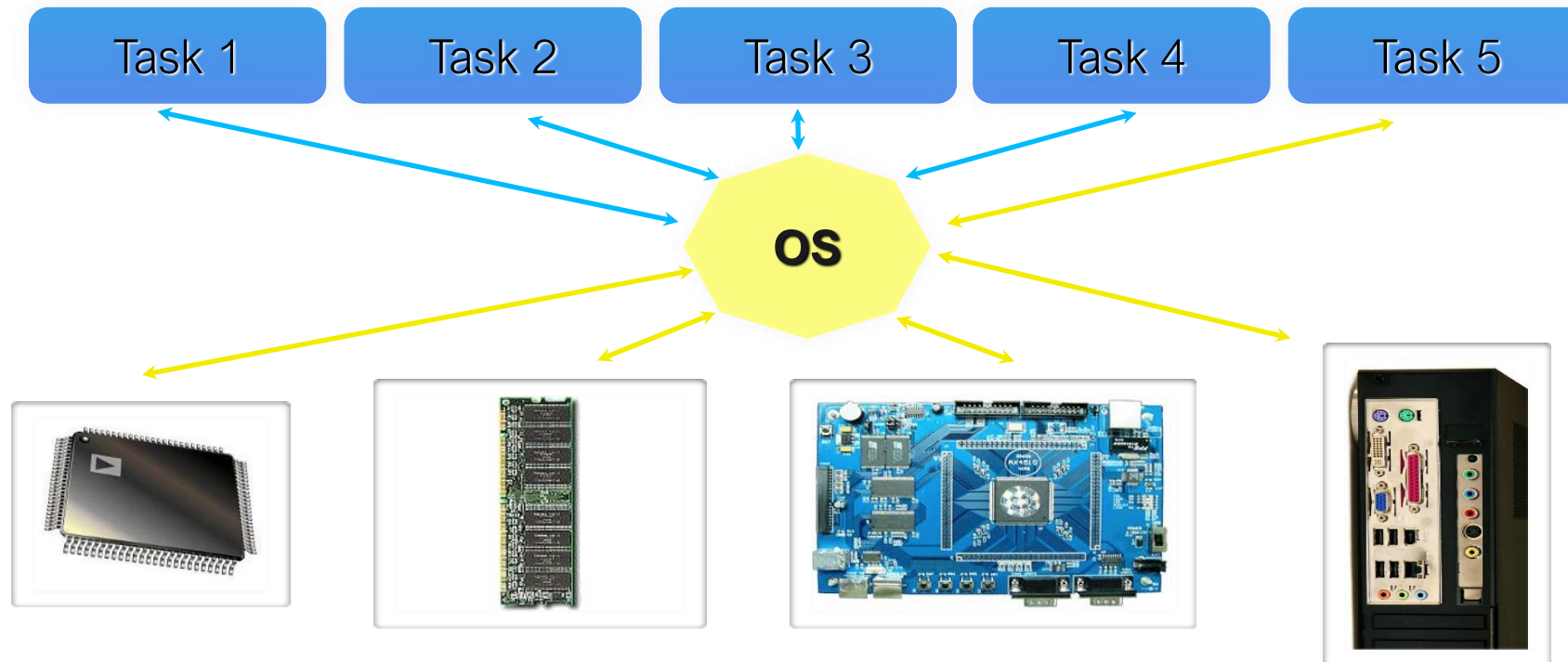
What it's all about: Why?



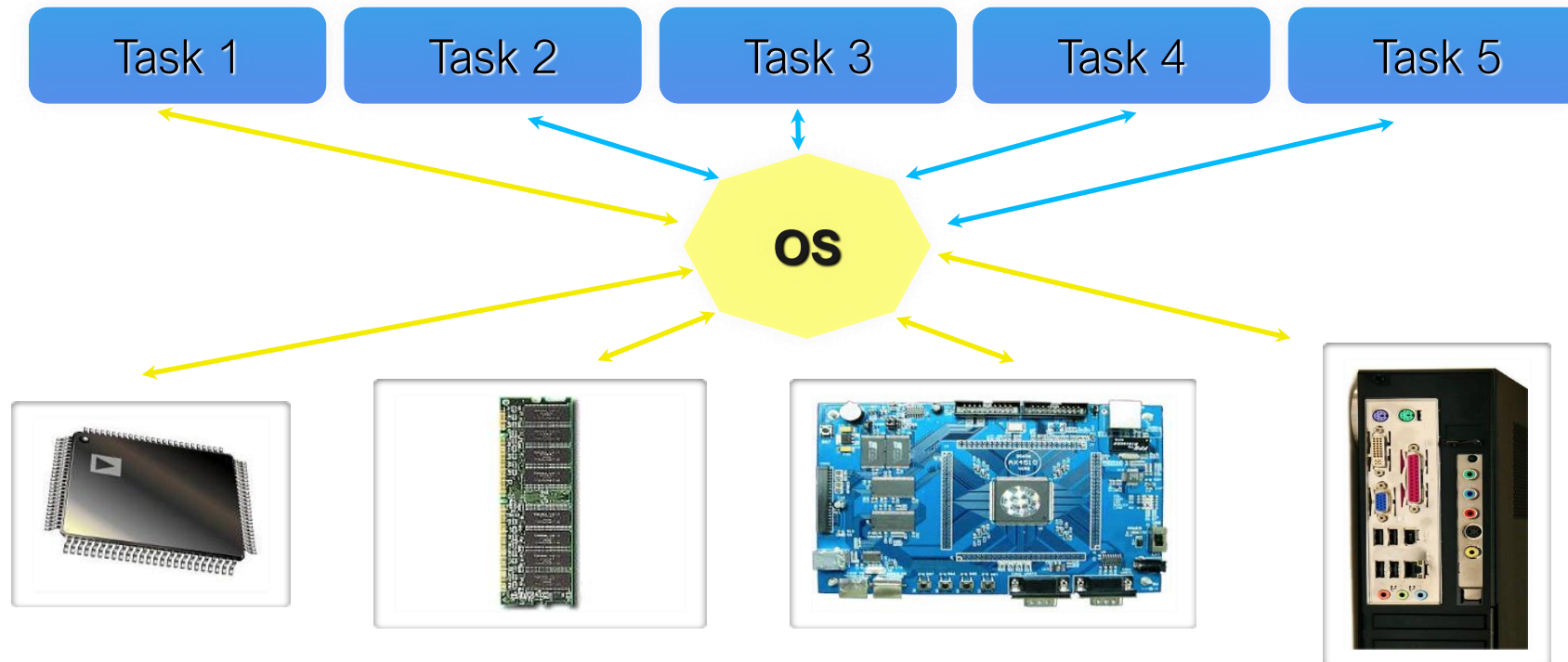
What it's all about: Why?



What it's all about: Why?



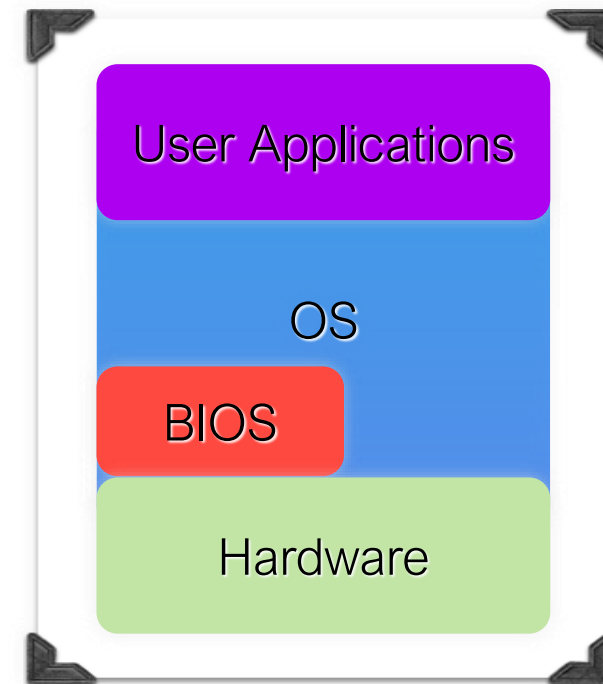
What it's all about: Why?



The Computer System

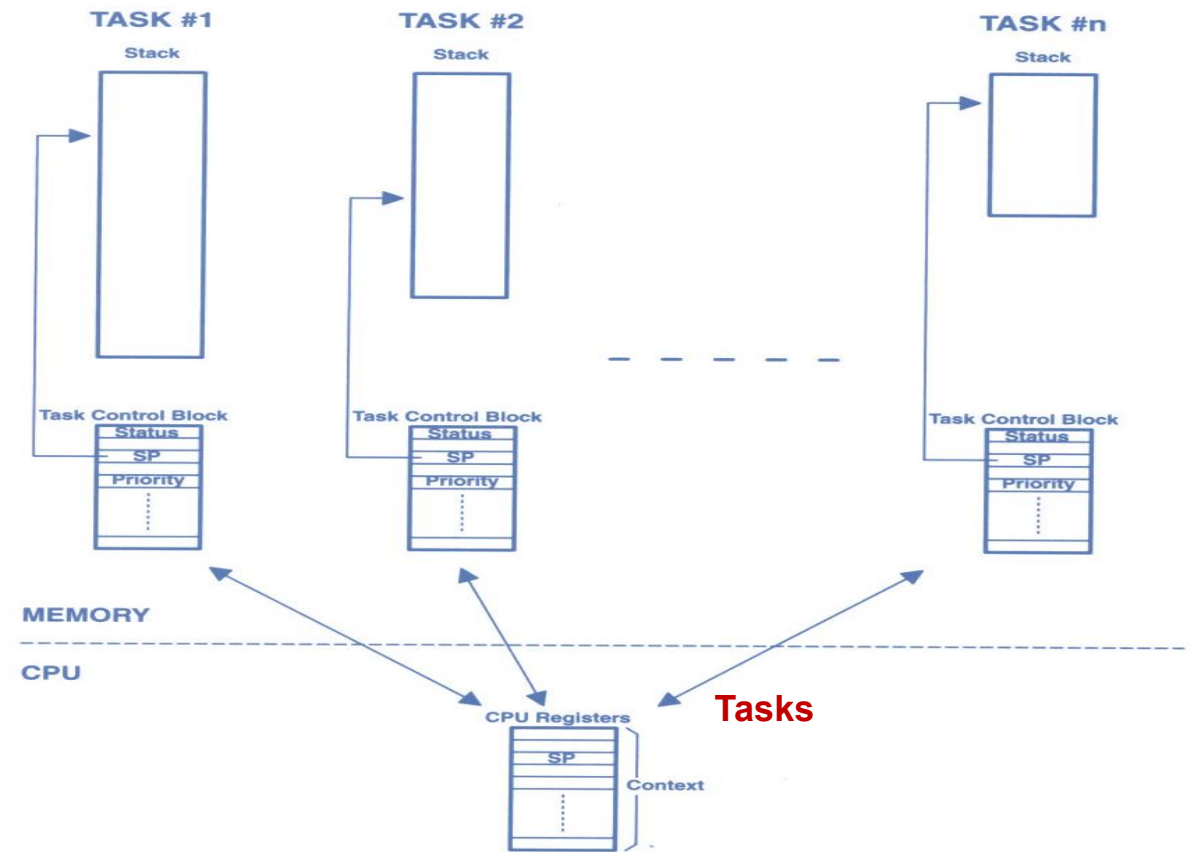
A computer system consists of three major components:

- Hardware
- Firmware
- Software



Operating System Concepts : Tasks

- Each task believes it has the CPU all to itself.
- Each task has a priority and its own stack.
- Each task does not complete



Operating System Concepts : Tasks

Multitasking

- Multitasking refers to the scheduling of multiple tasks and switching the CPU between several tasks.
- A single CPU is seen to switch its attention between several sequential tasks.
- Multitasking is like foreground/background with multiple backgrounds.
- Multitasking is an efficient use of the CPU and promotes a modular construction of applications.

OS Concepts: Implementing Multitasking

Time Slicing

Also known as ***time-slicing***.

When two or more tasks have the same priority, the kernel allows one task to run for a predetermined amount of time (***quantum***) and then selects another task of the same priority to run.



OS Concepts: Implementing Multitasking

- Also known as ***time-slicing***.
- When two or more tasks have the same priority, the kernel allows one task to run for a predetermined amount of time (***quantum***) and then selects another task of the same priority to run.



Assuming All Tasks have the same priority.

OS Concepts: Implementing Multitasking

Round Robin Scheduling

- Also known as ***time-slicing***.
- When two or more tasks have the same priority, the kernel allows one task to run for a predetermined amount of time (***quantum***) and then selects another task of the same priority to run.



Assuming All Tasks have the same priority.

OS Concepts: Implementing Multitasking

Round Robin Scheduling

- Also known as ***time-slicing***.
- When two or more tasks have the same priority, the kernel allows one task to run for a predetermined amount of time (***quantum***) and then selects another task of the same priority to run.



Assuming All Tasks have the same priority.

OS Concepts: Implementing Multitasking

Round Robin Scheduling

- Also known as ***time-slicing***.
- When two or more tasks have the same priority, the kernel allows one task to run for a predetermined amount of time (***quantum***) and then selects another task of the same priority to run.



Assuming All Tasks have the same priority.

OS Concepts: Implementing Multitasking

Round Robin Scheduling

- Also known as ***time-slicing***.
- When two or more tasks have the same priority, the kernel allows one task to run for a predetermined amount of time (***quantum***) and then selects another task of the same priority to run.



Assuming All Tasks have the same priority.

OS Concepts : Terms

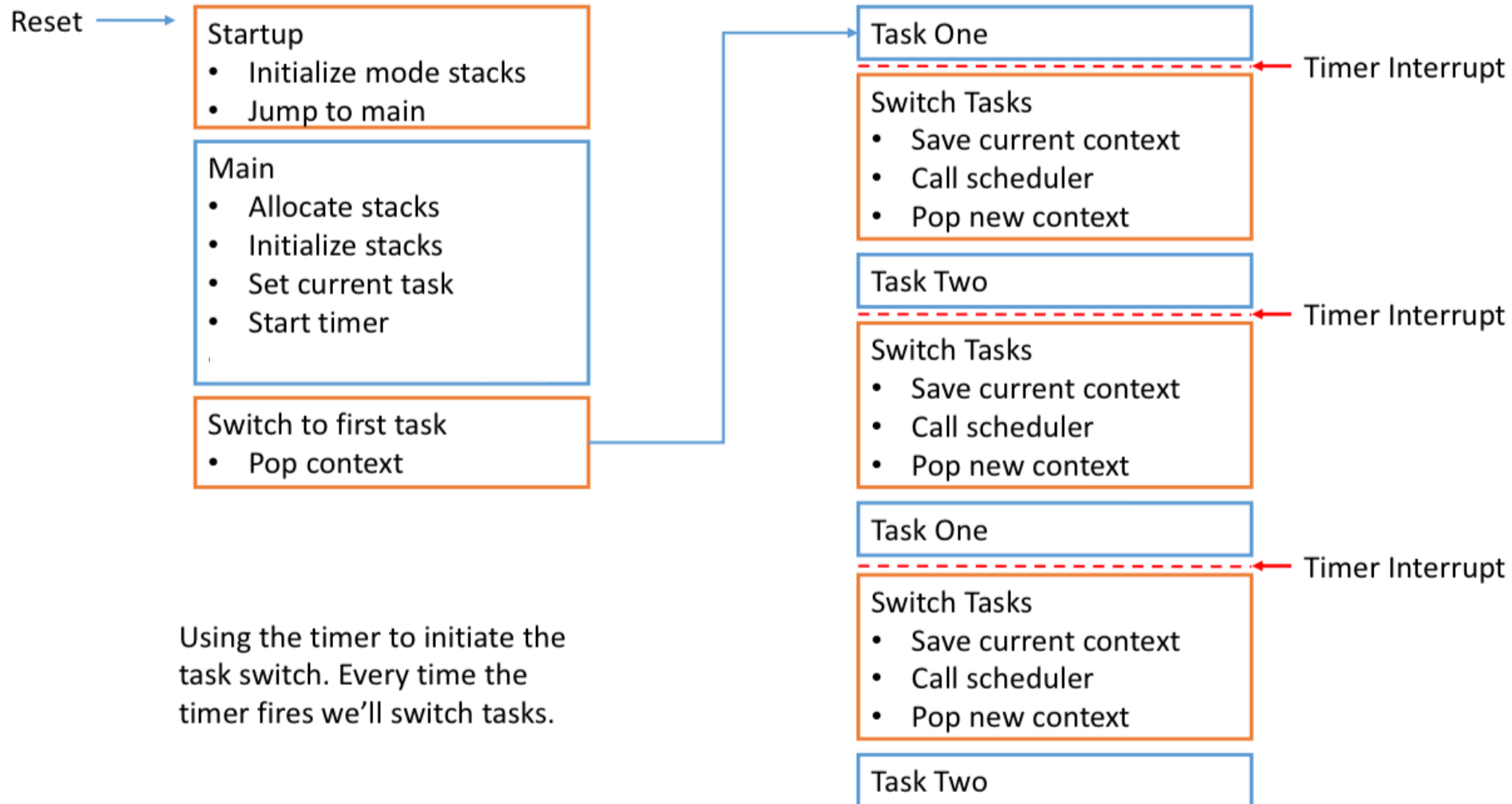
Resources

- A resource is any item that can be used by a task.
- These include memory, CPU, I/O (peripheral).

Shared Resources

- A shared resource is a resource than can be used by more than one task.
- The task must gain exclusive access to the resource in order to prevent data corruption (**mutual exclusion**).
- Mutual exclusion can be performed by disabling and enabling interrupts.

Multitasking: Switching Tasks

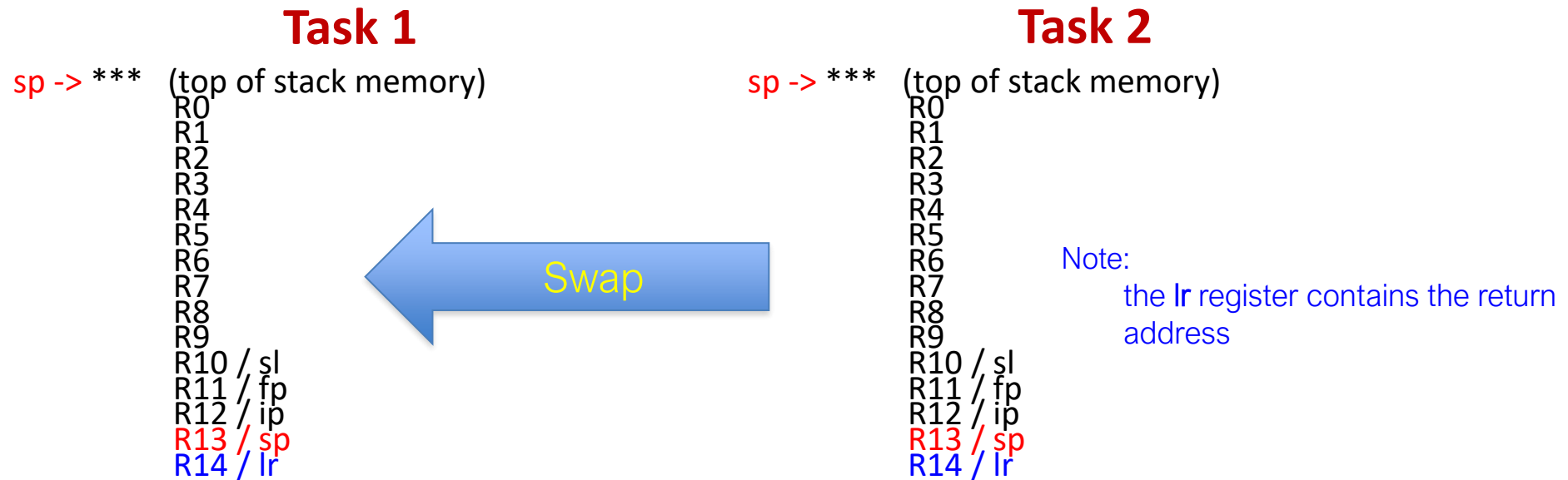


Multitasking: Switching Tasks

Context Switches

- When the CPU decides to run another task it stores the current CPU context in the current tasks context storage area.
- After the operation is complete the new tasks context is restored from its storage area onto the CPU and the new task resumes execution.
- This process is commonly referred to as **context switching** or **task switching**.
- Context switching adds overhead to the application. The time required to perform a context switch depends on the number of registers within the CPU as they all must be saved and restored.

Multitasking : Context Switch



On entry:

registers are stored on stack
Swap to new stack (this means new register values)

On exit:

registers are popped off stack
Program executes from new **lr** value.

An Example Header File

```
// RTOS Sizes
```

```
#define TLISTSIZE 6
```

```
#define STACKSIZE 100
```

```
// Structure holding the required task information
```

```
struct Task {
```

```
    // stack pointer into stack
```

```
    volatile unsigned int *stackpointer;
```

```
    // stack for the task
```

```
    volatile unsigned int stack[STACKSIZE];
```

```
    // task priority
```

```
    volatile unsigned int priority;
```

```
    // used to identify task in task list
```

```
    volatile unsigned int taskid;
```

```
} ;
```

```
// external functions
```

```
void task_1(void) ;
```

```
void task_2(void) ;
```

```
void init_scheduler(void) ;
```

```
unsigned int add_task(void (*newtask)(), unsigned int priority, unsigned int taskid) ;
```

```
unsigned int remove_task(unsigned int taskid) ;
```

```
void irq_handler(void) ;
```

```
// external variables
```

```
extern volatile struct Task TaskList[TLISTSIZE]; // scheduler task list
```

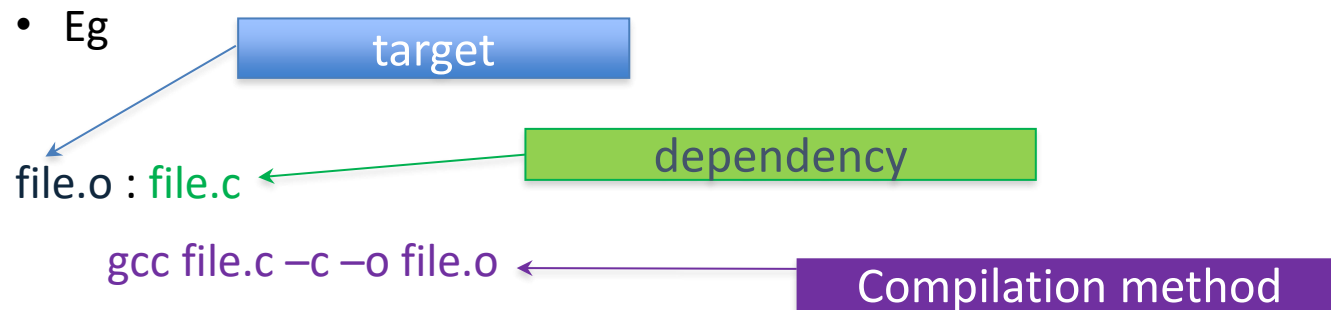
```
extern volatile unsigned int currenttask;
```

```
extern volatile unsigned int state, hex[16];
```

```
//end
```

Makefiles

- Makefiles contain a set of rules for compiling code
- Each rule contains a target & a dependency and a method to convert the dependency into the target



- If `file.c` is changed then the rule to create `file.o` will be performed.

Example Implementation : makefile

```
COPS = -Wall -O2 -nostdlib -nostartfiles -ffreestanding
```

```
SRCNAME = kernel
```

```
HEADERS = rtos.h rpi_io.h
```

```
IONAME = rpi_io
```

```
OBJFILES = vectors.o irq_ISR.o tasks.o init_scheduler.o add_task.o  
remove_task.o rpi_io.o
```

```
CFILES = irq_ISR.c tasks.c init_scheduler.c add_task.c remove_task.c  
rpi_io.h
```

```
gcc : clean vectors.o $(SRCNAME).bin
```

```
all : gcc
```

```
clean :
```

```
    rm -f *.bin
```

```
    rm -f *.list
```

```
$(SRCNAME) : $(SRCNAME).c $(HEADERS) $(OBJFILES)
```

```
    gcc $(COPS) -c $(SRCNAME).c -o $(SRCNAME).o
```

```
vectors.o : vectors.s
```

```
    as vectors.s -o vectors.o
```

```
irq_ISR.o : irq_ISR.c $(HEADERS)
```

```
    gcc $(COPS) -c irq_ISR.c -o irq_ISR.o
```

```
tasks.o : tasks.c $(HEADERS)
```

```
    gcc $(COPS) -c tasks.c -o tasks.o
```

```
init_scheduler.o : init_scheduler.c $(HEADERS)
```

```
    gcc $(COPS) -c init_scheduler.c -o init_scheduler.o
```

```
add_task.o : add_task.c $(HEADERS)
```

```
    gcc $(COPS) -c add_task.c -o add_task.o
```

```
remove_task.o : remove_task.c $(HEADERS)
```

```
    gcc $(COPS) -c remove_task.c -o remove_task.o
```